DEEP LEARNING (BCS714A)

Question Bank – Modules 3, 4 and 5
Model Answers in VTU Examination Format

Acharya Institute of Technology · Dept. of AI & ML
18 questions · each answer is 1–2 pages · with labelled diagrams, equations and worked numericals

Module	Questions
Module 3 (CNN)	Q.1 – Q.4
Module 4 (RNN / LSTM / Recursive nets)	Q.6 – Q.12
Module 5 (NLP, metrics, embeddings)	Q.12 – Q.18 (as numbered in the question bank)

Sources: module textbooks supplied (Goodfellow et al., *Deep Learning*, Ch. 9 & 10; Krohn et al., *Deep Learning Illustrated*, Ch. 11). Equations, figures' ideas and the numerical examples follow these books. Question 2 of the bank does not print its matrix, so a sample matrix is assumed and stated in the answer.

MODULE 3 · Q.1

Apply your understanding of sparse interactions, parameter sharing, and equivariance to explain how convolution improves the performance of image recognition models.

Answer:

Introduction. Convolution is a linear operation in which a small kernel K slides over the input I. In deep-learning libraries it is implemented as cross-correlation:

$$S(i, j) = (I * K)(i, j) = \Sigma_m \Sigma_n I(i + m, j + n) \cdot K(m, n)$$

A fully-connected layer multiplies the input by a weight matrix, so every output unit interacts with every input unit. Convolution replaces this with three ideas that make image models smaller, faster and more accurate: **sparse interactions, parameter sharing and equivariant representations**.

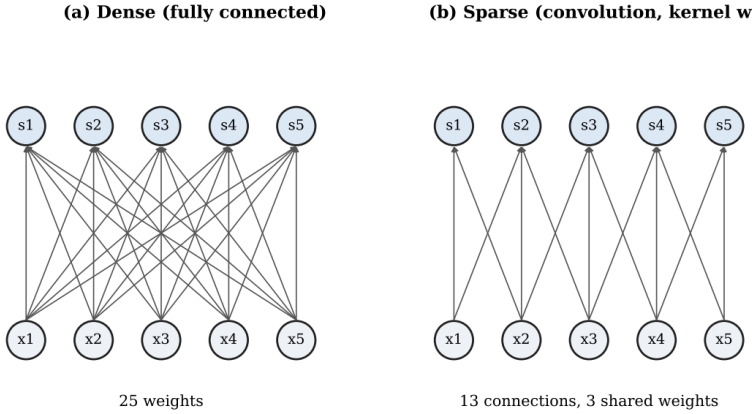


Fig. 1: Dense (matrix multiplication) vs sparse (convolution) connectivity

1. Sparse interactions (sparse connectivity / sparse weights)

- The kernel is much **smaller than the input**. A 1000×1000 image has millions of pixels, but meaningful features such as edges occupy only tens of pixels, so the kernel is only a few pixels wide.
- With m inputs and n outputs, matrix multiplication needs **O(m × n)** parameters and run-time; if each output is limited to k connections, only **O(k × n)** are needed (k □ m).
- **Example (textbook):** to detect vertical edges in a 320×280 image, convolution with a 2-element kernel needs 319×280×3 = **267,960** floating-point operations; an equivalent dense matrix would have 320×280×319×280 ≈ **8 billion** entries – about **4 billion times** more costly.
- In deeper layers a unit is *indirectly* connected to a larger region of the image (its **receptive field** grows), so complex features are built from simple ones.

2. Parameter sharing (tied weights)

- In a dense layer each weight is used exactly once. In a CNN the **same kernel weights are applied at every position** of the input, so the layer learns one set of weights instead of a separate set for every location.
- Storage falls from $m \times n$ to k parameters. Example: a 224×224×3 input fully connected to 1000 units needs 150,528 × 1000 ≈ **150 million** weights, whereas 64 filters of size 3×3×3 need only **1,728** weights.
- Fewer parameters → **less over-fitting**, lower memory, and better statistical efficiency (every image position provides training signal for the same filter).

3. Equivariance to translation

A function f is equivariant to a transformation g if **f(g(x)) = g(f(x))**. Because the same kernel is used everywhere, shifting the input shifts the output feature map by the same amount. If an object (or an edge) moves in the image, its representation moves with it; the detector does not have to be relearned for every position. In time-series data it produces a timeline showing *when* a feature appeared. (Convolution is not naturally equivariant to scale or rotation – other mechanisms are needed.) Pooling adds approximate **invariance** to small translations on top of this.

Comparison

Property	Fully-connected layer	Convolution layer
Connectivity	Every output ↔ every input	Local (kernel size k), sparse
Parameters	m × n	k (shared across positions)
Translation handling	Must learn each position separately	Equivariant (same detector everywhere)
Over-fitting risk / memory	High	Low

How these improve image-recognition performance

- **Efficiency:** fewer multiplications and parameters enable deeper networks on large images.
- **Better generalisation:** the detector learned in one place works in all places; less data is needed.
- **Hierarchical features:** edges → textures → parts → objects, built from small local, shared filters.
- **Robustness:** equivariance + pooling give tolerance to object position.

Conclusion: sparse interactions cut computation, parameter sharing cuts memory and over-fitting, and equivariance makes recognition independent of where the object appears – together these make CNNs the standard model for image recognition.

MODULE 3 · Q.2

Apply max pooling with a 2×2 window and stride 1 to the given convolution + ReLU output matrix, and reshape the resulting matrix into a one-dimensional vector for the next stage of the CNN workflow.

Answer:

Note: the question bank does not print the matrix. The following 4×4 convolution + ReLU output is **assumed**; if your paper gives another matrix, apply exactly the same steps.

$$\textit{Input (after Conv + ReLU)} = [[1, 3, 2, 1], [4, 6, 5, 2], [3, 2, 8, 4], [1, 5, 3, 7]]$$

Concept

Pooling replaces the output at a location by a summary statistic of the nearby outputs. **Max pooling** (Zhou & Chellappa, 1988) reports the maximum value in each rectangular neighbourhood. It makes the representation approximately **invariant to small translations**, reduces size and computation, and keeps the strongest activation. In a CNN the usual order is: **Convolution → ReLU (detector) → Pooling → Flatten → Fully-connected → Softmax**.

Step 1 – Output size

$$\textit{Output} = \lfloor (n - f) / s \rfloor + 1 = \lfloor (4 - 2) / 1 \rfloor + 1 = 3 \rightarrow 3 \times 3 \textit{ matrix}$$

Window f = 2×2, stride s = 1 (the window moves one cell at a time, so windows overlap), no padding.

Step 2 – Slide the window and take the maximum

Window position (rows, cols)	Elements	Max
(1-2, 1-2)	1, 3, 4, 6	**6**
(1-2, 2-3)	3, 2, 6, 5	**6**
(1-2, 3-4)	2, 1, 5, 2	**5**
(2-3, 1-2)	4, 6, 3, 2	**6**
(2-3, 2-3)	6, 5, 2, 8	**8**
(2-3, 3-4)	5, 2, 8, 4	**8**
(3-4, 1-2)	3, 2, 1, 5	**5**
(3-4, 2-3)	2, 8, 5, 3	**8**
(3-4, 3-4)	8, 4, 3, 7	**8**

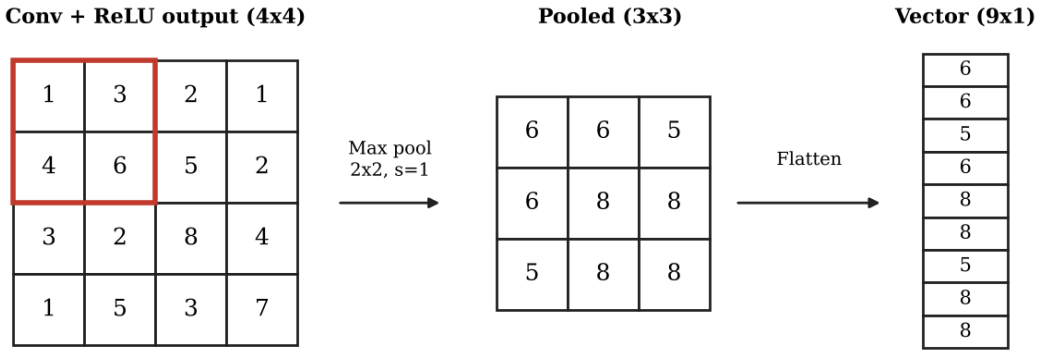


Fig. 2: Max pooling (2×2, stride 1) and flattening

Step 3 – Pooled feature map (3 × 3)

$$[[6, 6, 5], [6, 8, 8], [5, 8, 8]]$$

Step 4 – Reshape (flatten) to a 1-D vector

Flattening reads the matrix **row by row** (row-major order) and converts the 3×3 map into a 9×1 vector – the fixed-length input required by the fully-connected (dense) layer:

$$\textit{Flattened vector} = [6, 6, 5, 6, 8, 8, 5, 8, 8]^T \textit{ (size } 3 \times 3 = 9)$$

Observations

- Dimension reduced from 4×4 = 16 values to 9; the strongest activations (8 and 6) are preserved.
- Stride 1 gives overlapping windows – more information retained than stride 2 (which would give only a 2×2 map [[6,5],[5,8]]).

- Small shifts of a feature inside a window leave the pooled output unchanged (translation invariance); pooling has **no learnable parameters**.

Code (Keras)

```
model = Sequential([
    Conv2D(8, (3,3), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D(pool_size=(2,2), strides=1),
    Flatten(),                 # 2-D feature map -> 1-D vector
    Dense(10, activation='softmax')])
```

MODULE 3 · Q.3

Apply your understanding of CNNs to identify and handle different input data formats in a real-world application.

Answer:

Introduction. The input of a CNN is a **tensor**. Its *dimensionality* (1-D, 2-D, 3-D) depends on how many axes carry spatial/temporal structure, and its *channels* carry different quantities observed at each point. Convolution works with all of them because the kernel simply slides over the structured axes and sums over channels.

Types of input data handled by convolution (Goodfellow, §9.7)

Data type	Single channel	Multi-channel
1-D	Audio waveform – one amplitude per time step (1-D conv over time)	Skeleton animation data – joint angles of a 3-D character at each time step (each angle = a channel)
2-D	Audio pre-processed with Fourier transform (spectrogram): axes = time × frequency; X-ray image (grey)	Colour image – R, G, B channels; kernel moves over height and width
3-D	Volumetric data such as CT / MRI scans (x, y, z)	Colour video – time + 2-D space + RGB channels

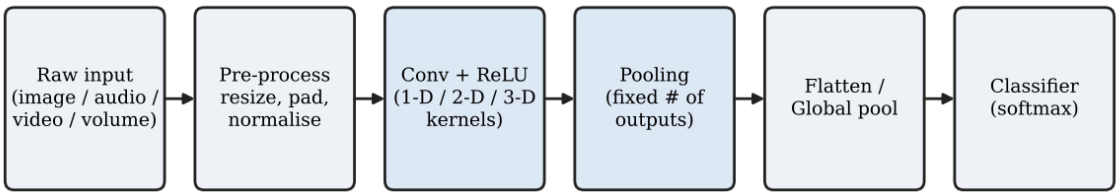


Fig. 3: Generic CNN workflow for handling heterogeneous inputs

Real-world application: AI-assisted medical diagnosis

A hospital system receives (i) **ECG signals** (1-D), (ii) **chest X-rays** (2-D, grey), (iii) **CT volumes** (3-D) and (iv) **endoscopy videos** (3-D, colour). One CNN family handles all by choosing kernel dimensionality to match the data:

Input	Tensor shape (batch, ...)	Layer used	Handling
ECG	(N, time, leads)	Conv1D	Leads = channels; normalise amplitude, resample to common rate
X-ray	(N, H, W, 1)	Conv2D	Resize / crop, intensity normalisation, augmentation (flip, rotate)
CT scan	(N, D, H, W, 1)	Conv3D	Resample voxel spacing, window intensities, patch-wise processing
Video	(N, T, H, W, 3)	Conv3D / Conv2D+LSTM	Frame sampling, RGB scaling to [0,1]

Handling inputs of varying size

- Ordinary matrix multiplication needs a fixed-size input, but **convolution can be applied a different number of times**, so the output simply grows with the input.
- For classification a fixed-size vector is still needed: the pooling region is scaled in proportion to the input size so the classifier always receives the same number of summary statistics (or use **global average / max pooling**).
- Alternatively **resize, crop or zero-pad** to a standard size; use **padding** (valid / same) to control feature-map size.
- Convolution on variable-size data is meaningful only when the size differs because the same kind of measurement is repeated (e.g. recordings of different length, images of different width).

General steps to handle any input format

- **Identify** the dimensionality (1-D/2-D/3-D) and the channels; choose Conv1D/2D/3D accordingly and set *input_shape* (channels-last).
- **Pre-process:** normalise/standardise, resize or pad, convert (e.g. waveform → spectrogram), augment.
- **Convolution + ReLU + pooling** blocks extract hierarchical features; **flatten/global pooling** produces a fixed vector.
- **Dense + softmax/sigmoid** gives the diagnosis; train with back-propagation and validate.

Conclusion: by matching kernel dimensionality to the data format and normalising/padding the input, one CNN design can process audio, images, volumes and video in a real application.

MODULE 3 · Q.4

Explain the variants of the basic convolution function with examples.

Answer:

Introduction. In practice a convolutional layer is not one single convolution: it applies **many kernels in parallel** to a multi-channel input (an image with colour channels, or the feature maps of the previous layer). The basic multi-channel convolution with input V (channel l, row, column) and 4-D kernel K (output channel i, input channel l, row offset m, column offset n) is:

$$Z_{i,j,k} = \Sigma_{l,m,n}\, V_{l,\, j+m-1,\, k+n-1} \cdot K_{i,l,m,n}$$

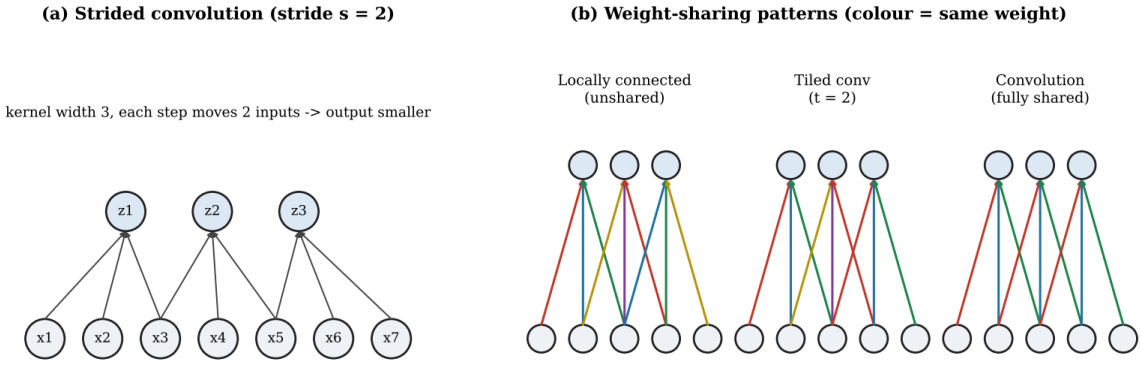


Fig. 4: (a) Strided convolution, (b) locally connected, tiled and standard convolution – same colour = same weight

1. Strided convolution (down-sampled convolution)

To reduce computation we may skip positions: the kernel moves s pixels at a time (stride s).

$$Z_{i,j,k} = c(K, V, s) = \Sigma_{l,m,n}\, V_{l,(j-1)\times s+m,(k-1)\times s+n} \cdot K_{i,l,m,n}$$

It is equivalent to unit-stride convolution followed by down-sampling, but cheaper because the skipped values are never computed. **Output size = ⌊(W − K + 2P)/S⌋ + 1.** Example: input width 7, kernel 3, stride 2, no padding → ⌊(7−3)/2⌋+1 = **3** outputs (Fig. 4a). Separate strides may be used in each direction.

2. Zero padding

Without padding the representation shrinks by (k − 1) at every layer, limiting depth. Three standard settings (for a 5×5 input and 3×3 kernel):

Type	Padding	Output size	Remark
Valid	None	3 × 3 (shrinks by k−1)	Kernel stays inside the image; fewer border effects but small output
Same	k−1 zeros in total (1 each side)	5 × 5 (= input)	Allows arbitrarily deep networks; border pixels under-represented
Full	k−1 zeros on every side	7 × 7 (grows by k−1)	Every pixel visited k times; output kernel-edge units see few real pixels

3. Unshared convolution (locally connected layer)

Connectivity is the same as convolution (small receptive field) but **every position has its own weights** – no sharing. With 6-D weight tensor W:

$$Z_{i,j,k} = \Sigma_{l,m,n}\, V_{l,j+m-1,k+n-1} \cdot w_{i,j,k,l,m,n}$$

Use: when a feature is expected only in a particular place, e.g. in face images the mouth detector is needed only in the lower half. **Cost:** far more parameters; no translation equivariance.

4. Tiled convolution

A compromise between convolution and locally connected layers: a **set of t kernels is rotated** as the window moves, i.e. weights are shared only between locations t steps apart:

$$Z_{i,j,k} = \Sigma_{l,m,n}\, V_{l,j+m-1,k+n-1} \cdot K_{i,l,m,n,j\%t+1,k\%t+1}$$

Memory grows only by a factor t (not by output size). Example: t = 2 alternates kernels K1, K2, K1, K2… (Fig. 4b). If t equals the output width it becomes a locally connected layer; t = 1 gives ordinary convolution.

5. Other variants

- **Channel-restricted connectivity:** each output channel is connected only to a subset of input channels – fewer parameters and computation.

- **Convolution on 1-D / 3-D data** (Conv1D, Conv3D) and **multi-channel** operation as in the basic equation above.
- **Pooling with stride / down-sampling** as an alternative way to reduce spatial size.

Summary

Variant	Weight sharing	Parameters	Main benefit
Standard	Full	Fewest	Equivariance, efficiency
Strided	Full	Same as standard	Smaller output, lower cost
Locally connected	None	Most	Position-specific features
Tiled (t)	Every t-th position	t × standard	Compromise

MODULE 4 · Q.6

Explain the architecture and working principle of a Recurrent Neural Network with a neat diagram. Given: x1 = 1, x2 = 0.5, h0 = 0, Wx = 0.5, Wh = 0.4, bh = 0, Wy = 1.2, by = 0 – calculate (a) h1, (b) h2 and the output y2.

Answer:

Introduction. A Recurrent Neural Network (RNN) is a neural network for processing a **sequence x(1), …, x(τ)**. Unlike a feed-forward network it has a **feedback (recurrent) connection** – the hidden state at time *t* depends on the current input and on the previous hidden state, so the network has memory of the past. The same parameters are used at every time step (**parameter sharing**), which lets the model generalise across positions in time and handle sequences of any length.

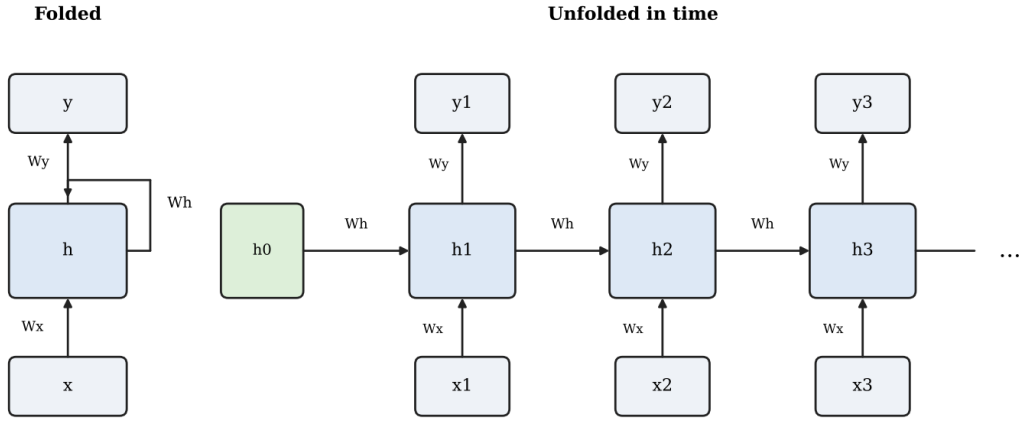


Fig. 5: RNN – folded (left) and unfolded computational graph (right)

Architecture

- **Input layer:** x(t) at each time step, connected to the hidden layer through weights **U (W_x)**.
- **Hidden layer:** state h(t) = memory; recurrent weights **W (W_h)** connect h(t−1) to h(t).
- **Output layer:** y(t) obtained from h(t) through **V (W_y)**; the loss $L = \Sigma_t L(t)$ compares y(t) with the target.
- **Unfolding** the loop over τ steps gives a deep feed-forward graph with **shared U, W, V**.

Working principle (forward propagation)

$$a(t) = b + W\,h(t-1) + U\,x(t);\; h(t) = \tanh(a(t));\; o(t) = c + V\,h(t);\; \hat{y}(t) = softmax(o(t))$$

In the notation of this question: $\mathbf{h_t = \tanh(W_x \cdot x_t + W_h \cdot h_{t-1} + b_h)}$ and $\mathbf{y_t = W_y \cdot h_t + b_y}$. Training uses **back-propagation through time (BPTT)**: the gradient is propagated backwards through the unrolled graph; its cost is O(τ) in time and memory and it cannot be parallelised across time. Long sequences suffer from **vanishing/exploding gradients**, which is why LSTM/GRU were introduced.

Numerical solution

(a) First hidden state

$$h_1 = \tanh(W_x x_1 + W_h h_0 + b_h) = \tanh(0.5\times1 + 0.4\times0 + 0) = \tanh(0.5) = \mathbf{0.4621}$$

(b) Second hidden state

$$h_2 = \tanh(0.5\times0.5 + 0.4\times0.4621 + 0) = \tanh(0.25 + 0.1848) = \tanh(0.4348) = \mathbf{0.4094}$$

(c) Output at the second time step

$$y_2 = W_y h_2 + b_y = 1.2 \times 0.4094 + 0 = \mathbf{0.4912}$$

t	x_t	h_{t-1}	$h_t = \tanh(0.5x_t + 0.4h_{t-1})$	$y_t = 1.2h_t$
1	1	0	0.4621	0.5545
2	0.5	0.4621	0.4094	0.4912

Types and applications

- Many-to-many (POS tagging, speech), many-to-one (sentiment), one-to-many (image captioning), sequence-to-sequence (translation).
- Applications: language modelling, machine translation, speech recognition, handwriting recognition, time-series forecasting.

MODULE 4 · Q.7

Demonstrate the working of a Bidirectional Recurrent Neural Network (BiRNN) with a neat diagram.

Answer:

Introduction. A standard (causal) RNN computes the state h(t) using only the **past** inputs x(1…t). In many tasks the correct output at time *t* depends on the **entire** input sequence: in speech recognition the interpretation of the current sound may depend on the next few phonemes and even on the next few words because of linguistic context; in handwriting recognition the next letters help decide the current one. **Bidirectional RNNs** (Schuster & Paliwal, 1997) were invented to meet this need.

Architecture

A BiRNN combines **two independent RNNs** that read the same sequence in opposite directions:

- **Forward RNN** – state **h(t)** moves forward through time (t = 1 … τ); it summarises the past x(1…t).
- **Backward RNN** – state **g(t)** moves backward through time (t = τ … 1); it summarises the future x(t…τ).
- The **output unit** at time t receives both h(t) and g(t), so it depends on both past and future but is most sensitive to the input values around time t – without a fixed-size window or look-ahead buffer.

$$y(t) = \text{softmax}(V[h(t)\ ;\ g(t)] + c)$$

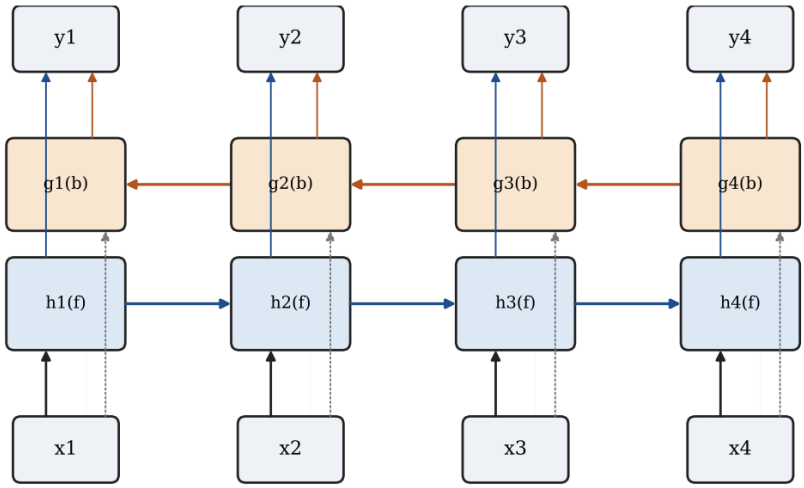


Fig. 6: Computation of a bidirectional RNN (forward states h, backward states g, outputs y)

Working – equations

$$h(t) = \tanh(b_f + W_f\, h(t-1) + U_f\, x(t)) \text{ (forward pass, } t = 1 \to \tau)$$

$$g(t) = \tanh(b_b + W_b\, g(t+1) + U_b\, x(t)) \text{ (backward pass, } t = \tau \to 1)$$

$$o(t) = c + V\,[h(t)\ ;\ g(t)],\ \hat{y}(t) = \text{softmax}(o(t))$$

Step by step: (1) read the whole sequence; (2) run the forward RNN left→right and store all h(t); (3) run the backward RNN right→left and store all g(t); (4) concatenate h(t) and g(t) at every t; (5) pass through the output layer to get ŷ(t); (6) training: the loss L = Σ L(t) is back-propagated (BPTT) through both directions, and the forward and backward parameters are updated together.

Example

For the sentence *"He said Teddy bears are on sale"* vs *"He said Teddy Roosevelt was president"*, deciding whether "Teddy" is a toy or a person needs the **following** words. The forward RNN cannot see them; the backward RNN supplies this information.

Extensions

- The idea extends to **2-D inputs (images)** with four RNNs (up, down, left, right); each point (i, j) then gets information from both local and long-range context. Compared with CNNs it is usually more expensive but allows long-range lateral interactions between features in the same feature map.
- Bi-directional **LSTM / GRU** (Bi-LSTM) replaces the simple cells – the standard choice in modern NLP.

Advantages and limitations

Advantages	Limitations
Uses both past and future context → higher accuracy	Needs the **complete** sequence – not suitable for real-time / online prediction
Better for tagging, speech, handwriting, bioinformatics	Twice the parameters and computation of a unidirectional RNN

MODULE 4 · Q.8

Differentiate between Recurrent Neural Networks and Recursive Neural Networks.

Answer:

Introduction. Both models apply the **same set of weights repeatedly** to build a representation of a variable-size input. A **recurrent** neural network does so along a **chain** (time); a **recursive** neural network (Pollack, 1990) generalises the computational graph from a chain to a **tree**. To avoid confusion, the textbook advises not abbreviating "recursive neural network" as "RNN".

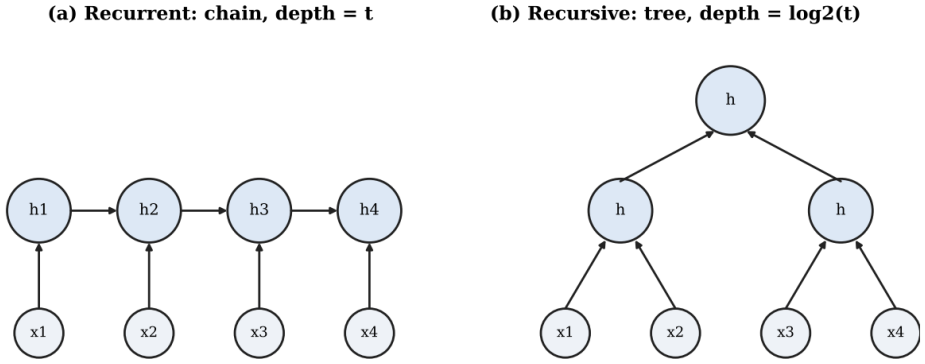


Fig. 7: (a) Recurrent network – chain of depth t; (b) recursive network – tree of depth $\log_2 t$

Comparison

Basis	Recurrent Neural Network	Recursive Neural Network
Structure of computation graph	Linear **chain** unrolled over time	**Tree** (usually binary) over the input
Input	Sequence x(1), x(2), …, x(τ)	Tree / hierarchical data, or a sequence with a chosen tree structure (e.g. parse tree)
Direction of processing	Left→right in time (one input per step)	Bottom-up: leaves → root; two children combine into a parent
Composition	h(t) = f(h(t−1), x(t))	$h_{parent} = f(h_{left}, h_{right}) = \tanh(W[h_l; h_r] + b)$
Depth for a sequence of length τ	**O(τ)** – long paths, vanishing/exploding gradient	**O(log τ)** for a balanced tree – shorter paths, helps long-term dependencies
Weights	U, W, V shared across time	Same W shared at every internal node of the tree
Structure known in advance?	Not needed – order of time defines it	Tree must be fixed (balanced tree), given by an external parser, or learned
Training algorithm	Back-propagation through time (BPTT)	Back-propagation through structure (BPTS)
Parallelism	Sequential, hard to parallelise	Nodes at same level can be computed in parallel
Special case	—	A recurrent net is a recursive net whose tree is a chain
Typical applications	Language modelling, speech recognition, time-series forecasting, translation	Parsing natural language, sentiment over parse trees, scene parsing, logical/arithmetic expression trees, structured data (molecules, XML)
Main difficulty	Capturing long-term dependencies	Choosing the best tree structure

Key points

- Recurrent nets model *temporal* dependencies; recursive nets model *hierarchical / compositional* relationships.
- Because path length from any leaf to the root is O(log τ), a recursive net can reduce the vanishing-gradient problem; LSTM-style gates can also be used inside tree nodes (**Tree-LSTM**).
- Many variants exist: node-specific targets, tensor-based composition (neural tensor network), data-dependent trees.

Conclusion: recurrent networks process data in temporal order along a chain; recursive networks process hierarchically along a tree, giving shorter dependency paths but requiring tree-structured input.

MODULE 4 · Q.9

Explain the architecture and working principle of a Long Short-Term Memory (LSTM) network with a neat diagram.

Answer:

Introduction. Gradients propagated over many time steps in an ordinary RNN tend to **vanish** (or, rarely, explode), so a simple RNN cannot learn long-term dependencies. The **Long Short-Term Memory** (Hochreiter & Schmidhuber, 1997) introduced **gated, self-loop paths** through which gradients can flow for long durations. The weight of the self-loop is not fixed but **conditioned on the context and controlled by another hidden unit (a gate)**, so the time-scale of integration can change dynamically.

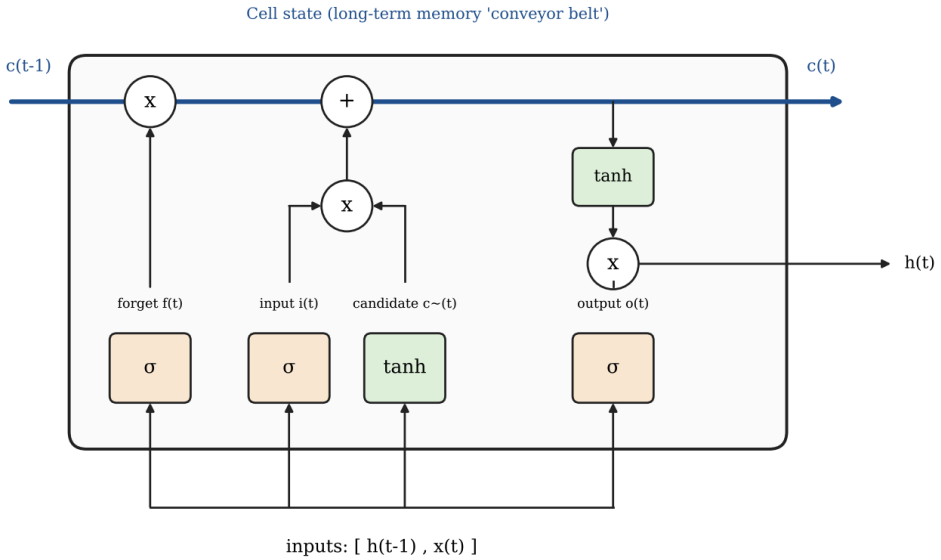


Fig. 8: Block diagram of an LSTM cell (σ = sigmoid gate, tanh = candidate/output squashing, × = element-wise multiplication)

Architecture – cell state and three gates

- **Cell state c(t)** (internal state $s_i(t)$ in the textbook): the linear self-loop – the long-term memory "conveyor belt".
- **Forget gate f(t):** decides what part of c(t−1) is kept (0 = erase, 1 = keep).
- **Input (external input) gate i(t):** decides how much of the new candidate c̃(t) is written.
- **Output gate o(t) (q in the textbook):** decides how much of the cell state is exposed as the hidden state h(t); it can shut off the output.

Equations (x(t) = input, h(t−1) = previous hidden state)

$$f(t) = \sigma(W_f[h(t-1), x(t)] + b_f) \text{ (forget gate)}$$

$$i(t) = \sigma(W_i[h(t-1), x(t)] + b_i) \text{ (input gate)}$$

$$\tilde{c}(t) = \tanh(W_c[h(t-1), x(t)] + b_c) \text{ (candidate memory)}$$

$$c(t) = f(t) \odot c(t-1) + i(t) \odot \tilde{c}(t) \text{ (cell-state update)}$$

$$o(t) = \sigma(W_o[h(t-1), x(t)] + b_o);\ h(t) = o(t) \odot \tanh(c(t))$$

In the textbook's notation: $s_i(t) = f_i(t) \cdot s_i(t-1) + g_i(t) \cdot \sigma(b_i + \Sigma_j U_{ij} x_j(t) + \Sigma_j W_{ij} h_j(t-1))$, $h_i(t) = \tanh(s_i(t)) \cdot q_i(t)$, where f is the forget gate, g the external input gate and q the output gate.

Working principle (step by step)

- **Step 1 – Forget:** f(t) scales the old memory c(t−1).
- **Step 2 – Write:** i(t) selects how much of the candidate c̃(t) is added; the sum gives the new c(t).
- **Step 3 – Read:** o(t) filters tanh(c(t)) to produce h(t), which goes to the next layer and to the next time step.
- All gates are sigmoids (values 0–1) so they act as soft switches; the whole cell is differentiable and trained with BPTT.

Why LSTM solves the vanishing-gradient problem

The cell state is updated **additively** ($c(t) = f \odot c(t-1) + \ldots$). When $f \approx 1$ the gradient $\partial c(t)/\partial c(t-1) \approx 1$, so error signals flow through many steps without shrinking – the constant-error carousel. LSTMs have been highly successful in handwriting recognition and generation, speech recognition, machine translation, image captioning and parsing.

Variants

- **Peephole connections** let gates see the cell state; **GRU** merges forget/input gates into one update gate and merges cell and hidden state (fewer parameters).
- **Bi-LSTM** and **stacked LSTM** for richer context.

MODULE 4 · Q.10

Summarize how deep recurrent networks capture temporal dependencies compared with shallow RNNs.

Answer:

Introduction. The computation in most RNNs can be decomposed into three blocks of parameters and transformations: (1) **input → hidden**, (2) **hidden(t−1) → hidden(t)**, and (3) **hidden → output**. In a **shallow RNN** each block is a single affine transformation followed by a non-linearity (one layer). A **deep recurrent network** introduces depth in these blocks.

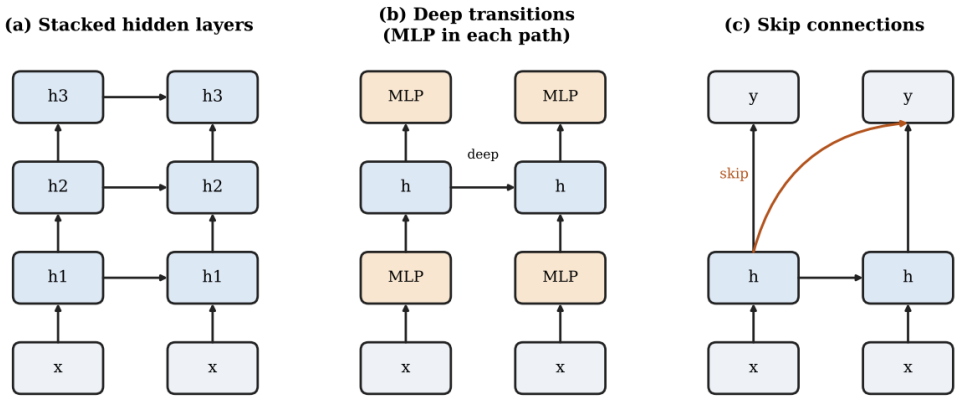


Fig. 9: Three ways of making an RNN deep – (a) stacked hidden layers, (b) deep transitions, (c) skip connections

Ways of adding depth (Goodfellow, Fig. 10.13)

- **(a) Hierarchical hidden states (stacked RNN):** several recurrent layers h^1, h^2, h^3 are stacked; the lower layer feeds the higher layer. Lower layers capture short, local patterns; higher layers capture longer, more abstract temporal structure – the same way depth helps in feed-forward networks.
- **(b) Deep transitions:** an MLP (deep, non-linear transformation) is inserted in the input→hidden, hidden→hidden and hidden→output paths, so the model can learn more complex state-transition functions and richer representations.
- **(c) Skip connections:** adding depth lengthens the shortest path between time steps (making optimisation harder). Direct connections from hidden state to a later one or to the output shorten these paths and ease gradient flow.

Comparison with shallow RNNs

Aspect	Shallow RNN	Deep recurrent network
Layers	One hidden recurrent layer; one-step transitions	Several recurrent layers / MLPs in each block
Representation	Simple, single level	Hierarchical: low-level to abstract temporal features, multiple time-scales
Capacity	Limited, needs wide hidden layer	Higher; the state can be transformed non-linearly more than once per step
Temporal dependencies	Struggles with complex or long-range patterns	Better at capturing both short-term details and longer-term structure; supports skip/ gated paths
Optimisation	Easier, but vanishing gradients over time	Harder (longer paths in both depth and time); remedies: skip connections, LSTM/GRU cells, gradient clipping, dropout, normalisation
Cost	Low	More parameters and computation
Performance	Adequate for small tasks	Experimentally (Graves et al., Pascanu et al.) better in speech, language modelling, translation

Summary

- Depth adds **representational power per time step** (more non-linear processing between input, state and output) and creates a **hierarchy of temporal abstraction**.
- The trade-off is a longer path from early inputs to later outputs (depth × time). Skip connections and gated units (LSTM/GRU) restore short gradient paths.

- In practice 2–4 stacked LSTM/GRU layers with dropout are used; stacked Bi-LSTM models outperform shallow ones (e.g. 94.9 vs 93.5 ROC AUC in the sentiment experiments of Module 5).

MODULE 4 · Q.11

Apply your understanding of Recursive Neural Networks to design a model of relationships in tree-structured data.

Answer:

Introduction. Many data sets are naturally **hierarchical**: parse trees of sentences, arithmetic or logical expressions, scene graphs of an image, XML/HTML DOM trees, molecules. A **recursive neural network** generalises the recurrent chain to a tree: the same composition function is applied at every internal node to combine the representations of its children into a representation of the parent, so the meaning of the whole is built from the meaning of its parts.

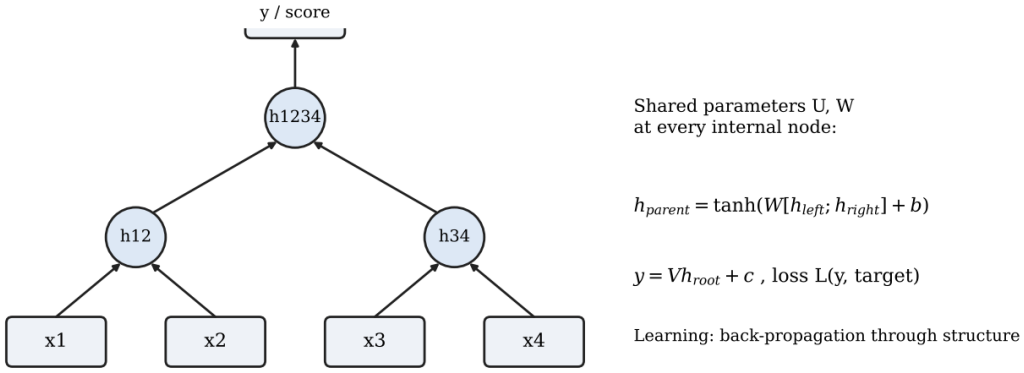


Fig. 10: Recursive network – shared parameters combine child representations bottom-up; a prediction is made at the root

Design steps

- **Step 1 – Choose the tree structure.** (i) Fixed, data-independent tree (e.g. balanced binary tree); (ii) structure supplied by an external method – e.g. the **parse tree** of a sentence from an NLP parser; (iii) structure **learned** by the model itself.
- **Step 2 – Represent the leaves.** Each leaf x_i is a vector – a word embedding, a pixel/segment feature vector, an atom feature vector.
- **Step 3 – Composition function (shared by all nodes).**

$$h_{parent} = tanh(\ W\ [h_{left}\ ;\ h_{right}] + b\ )$$

- **Step 4 – Model the relationship.** The weight matrix W learns how two children combine (e.g. adjective + noun, negation + word). For richer interaction use a **tensor / bilinear composition** (neural tensor network): h = tanh([a;b]^T $V^{[1:d]}$ [a;b] + W[a;b]). Data-dependent gates (Tree-LSTM) let the node control information flow.
- **Step 5 – Output.** Apply a classifier at the root (or at every node): $\hat{y}$ = softmax(V h_{root} + c). Targets may be attached to the whole tree (sentence sentiment) or to individual nodes (phrase labels).
- **Step 6 – Loss and training.** L = cross-entropy between $\hat{y}$ and target; gradients are computed by **back-propagation through structure (BPTS)** – the error flows from the root down to the leaves and the gradients from all nodes are summed because the weights are shared. Use SGD/Adam, dropout and regularisation.

Worked example – sentiment relation in a tree

For the phrase *“not very good”* with tree (not (very good)): h_1 = tanh(W[very ; good]) – an intensified positive meaning; h_2 = tanh(W[not ; h_1]) – the negation flips the polarity, giving a negative-sentiment vector at the root. A chain RNN must carry the word “not” over several steps, while the tree combines it directly with the phrase it modifies.

Advantages

- **Depth O(log τ)** for a balanced tree instead of τ for a chain – helps reduce vanishing/exploding gradients and long-term dependency problems.
- Explicitly models **compositional relationships**; variable-size input handled with fixed parameters (parameter sharing).
- Applications: sentence parsing and sentiment, paraphrase detection, image scene parsing, program/expression evaluation, molecule property prediction.

Challenges

- The best tree structure is an open question (parser errors propagate); batching trees of different shapes is harder than batching sequences.

MODULE 4 · Q.12

Implement a tree-based representation of "I study Deep Learning" using the Recursive Neural Network concept and contrast it with a Deep Recurrent Network.

Answer:

Step 1 – Obtain the tree. A syntactic parser gives S → NP(I) VP, VP → V(study) NP, NP → (Deep Learning). Binarised: **(I (study (Deep Learning)))**. The leaves are the word vectors x_I, x_{study}, x_{Deep}, $x_{Learning}$ (d-dimensional embeddings).

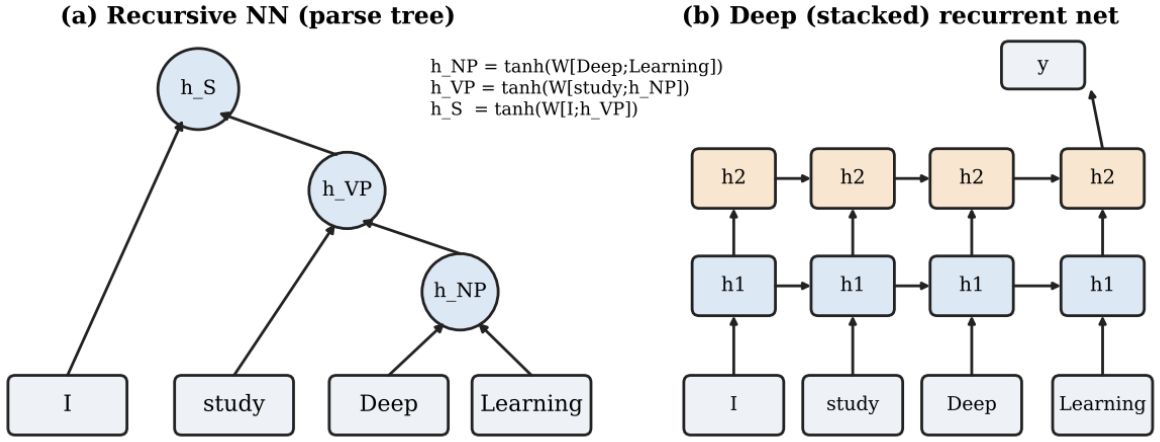


Fig. 11: (a) Recursive neural network over the parse tree of "I study Deep Learning"; (b) two-layer (deep) recurrent network over the same words

Step 2 – Bottom-up composition with shared weights W (d × 2d) and bias b

$$h_{NP} = tanh(\ W\ [x_{Deep}\ ;\ x_{Learning}] + b\ )\ \textit{(phrase "Deep Learning")}$$
$$h_{VP} = tanh(\ W\ [x_{study}\ ;\ h_{NP}] + b\ )\ \textit{(phrase "study Deep Learning")}$$
$$h_S = tanh(\ W\ [x_I\ ;\ h_{VP}] + b\ )\ \textit{(whole sentence)}$$
$$\hat{y} = softmax(\ V\ h_S + c\ )$$

Only **three** composition steps are needed and the same W is reused each time. The root vector h_S is the sentence representation used for classification (e.g. topic = "education"). Training: cross-entropy loss at the root, gradients flow back through $h_S \rightarrow h_{VP} \rightarrow h_{NP} \rightarrow$ leaves by back-propagation through structure.

Step 3 – Deep recurrent network on the same sentence

Words are read in order, one per time step, by L stacked recurrent layers (L = 2 in Fig. 11b):

$$h^1{}_t = tanh(U^1 x_t + W^1 h^1{}_{t-1} + b^1);\ h^2{}_t = tanh(U^2 h^1{}_t + W^2 h^2{}_{t-1} + b^2);\ \hat{y} = softmax(V\ h^2{}_4 + c)$$

Contrast

Feature	Recursive NN (tree)	Deep recurrent network
Structure	Binary parse tree: (I (study (Deep Learning)))	Chain I → study → Deep → Learning, with 2 stacked layers
Needs syntax / parser	Yes	No
Number of sequential composition steps	3 (tree depth = 3)	4 time steps × 2 layers
Longest path leaf → output	3 compositions: O(log τ) for balanced trees	Up to 4 time steps + 2 layers: O(τ + L)
What is composed	"Deep + Learning" first (a meaningful phrase), then verb phrase, then subject	Words are accumulated left to right; phrases are not explicit
Weights	One W shared across all nodes	Separate weights per layer, shared across time
Gradient algorithm	BPTS	BPTT
Strength	Explicit hierarchical relations; short paths	Simple, any sequence, no tree needed; deeper abstraction per step

Conclusion: the recursive net exploits the grammatical hierarchy of the sentence, whereas the deep recurrent net learns temporal/abstract structure from the raw word order.

MODULE 5 · Q.12

Demonstrate the common steps involved in preprocessing natural language data.

Answer:

Introduction. Natural-language text is noisy and high-dimensional. Pre-processing converts the raw corpus into clean, consistent **tokens** that can be embedded (word2vec/GloVe) and fed to a deep-learning model, which usually improves downstream accuracy.

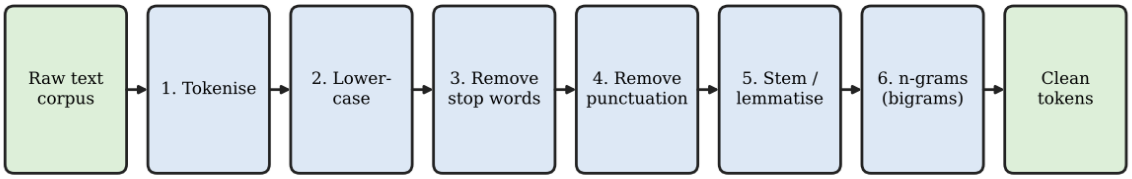


Fig. 12: Pipeline of natural-language preprocessing steps

Common preprocessing steps

Step	What it does	Example
1. Tokenisation	Splits the document into discrete elements (sentences, then words) called tokens – nltk *sent_tokenize*, *word_tokenize*	“She is happy.” → [She, is, happy, .]
2. Lower-casing	Converts all characters to lowercase; capitalisation (start of a sentence) carries no extra meaning	She → she
3. Removing stop words	Removes frequent words with little distinctive meaning (the, at, which, of). Lists are not universal; keep negations (didn’t, isn’t, wouldn’t) for sentiment tasks	the, of, at removed
4. Removing punctuation	Punctuation marks generally add little value	, . ! ? removed
5. Stemming (or lemmatisation)	Truncates words to their stem so related words share one token – useful for **small** corpora (more examples per token). Lemmatisation needs a reference vocabulary	house, housing → hous
6. Handling n-grams	Words that often co-occur are merged into one token (bigrams), found with gensim *Phrases* (min_count, threshold)	new, york → new_york

Python illustration (nltk + gensim)

```
from nltk import sent_tokenize, word_tokenize
from nltk.corpus import stopwords
from nltk.stem.porter import PorterStemmer
from gensim.models.phrases import Phraser, Phrases
import string
stop = stopwords.words('english') + list(string.punctuation)
stem = PorterStemmer()
sents = [word_tokenize(s) for s in sent_tokenize(raw_text)]             # 1 tokenise
clean = [[stem.stem(w.lower()) for w in s if w.lower() not in stop]    # 2-5
         for s in sents]
bigram = Phraser(Phrases(clean, min_count=32, threshold=64))           # 6 n-grams
clean_sents = [bigram[s] for s in clean]
```

Guidelines for choosing the steps

- Use all, some or none of the steps depending on the data and application: stemming helps a small corpus but hurts a large one; stop words may carry sentiment (e.g. in movie reviews they are *“indicative of sentiment”*, so Maas et al. kept them).
- The **larger the corpus**, the fewer preprocessing steps are helpful – with enough data the model can learn from the raw tokens.

- Very rare words and out-of-vocabulary words are handled with *min_count* thresholds (rare tokens dropped) or by using sub-word models such as fastText.
- Compare the model's performance with and without a step if unsure.

Result: a list of lists of clean tokens (*clean_sents*) – the input for word-vector creation with word2vec.

MODULE 5 · Q.13

Explain the confusion matrix for binary classification and derive the equations of accuracy, precision, recall and F1-score.

Answer:

Introduction. A binary classifier assigns each input to the positive class (1) or the negative class (0). The **confusion matrix** is a 2 × 2 table that compares the predicted class with the actual class – it shows how "confused" the model is and is the starting point for every evaluation metric (and the ROC AUC).

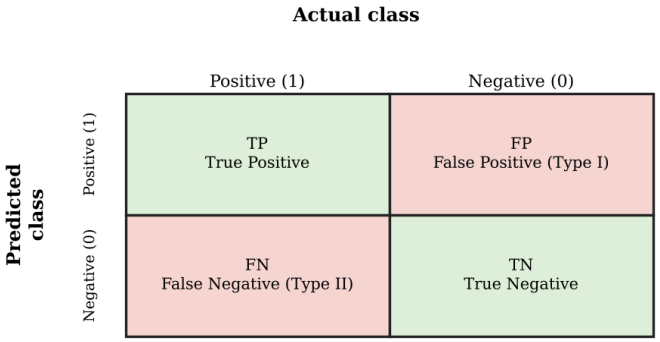


Fig. 13: Confusion matrix (rows = predicted class, columns = actual class)

The four outcomes (example: hot-dog / not-hot-dog classifier)

Term	Predicted	Actual	Meaning
TP – True Positive	1	1	Hot dog correctly identified as a hot dog
FP – False Positive (Type I error)	1	0	Not-hot-dog wrongly classified as a hot dog – model confused
FN – False Negative (Type II error)	0	1	Hot dog missed (called not-hot-dog) – model confused
TN – True Negative	0	0	Not-hot-dog correctly rejected

Derivation of the metrics

(1) Accuracy – fraction of all predictions that are correct. Correct predictions are TP + TN, all predictions are TP + TN + FP + FN:

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$

(2) Precision – of everything the model labelled positive (TP + FP), how many are really positive. It measures the *quality of positive predictions* (penalises FP):

$$Precision\ (P) = TP / (TP + FP)$$

(3) Recall (sensitivity, true positive rate) – of all actual positives (TP + FN), how many were found. It measures *completeness* (penalises FN):

$$Recall\ (R) = TP / (TP + FN)$$

(4) F1-score – precision and recall trade off against each other, so we combine them with the **harmonic mean** (which is dominated by the smaller value; the arithmetic mean could look good even if one is near 0):

$$1/F1 = ½\ (1/P + 1/R) = (P + R)/(2PR) \Longrightarrow F1 = 2PR/(P + R) = 2TP / (2TP + FP + FN)$$

(Substituting P = TP/(TP+FP) and R = TP/(TP+FN) and simplifying gives the last form.) Other useful rates: **Specificity = TN/(TN+FP)**; **False-positive rate = FP/(FP+TN)**.

Numerical example (TP = 40, FP = 10, FN = 20, TN = 30; N = 100)

Metric	Calculation	Value
Accuracy	(40+30)/100	0.70
Precision	40/(40+10)	0.80
Recall	40/(40+20)	0.667
F1-score	2×0.80×0.667/(0.80+0.667)	0.727

When to use which metric

- **Accuracy** is misleading for imbalanced data: a model that always predicts “not fraud” on 95 % genuine transactions has 95 % accuracy but 0 recall.
- **Precision** matters when false alarms are costly (spam filter); **recall** when misses are costly (cancer screening); **F1** when both matter and classes are imbalanced.
- Accuracy uses a single threshold ($\hat{y} > 0.5$); varying the threshold gives different matrices – leading to the ROC curve and AUC.

MODULE 5 · Q.14

Discuss the concept of attention mechanisms in NLP and explain how attention enables a decoder to focus on relevant input tokens during sequence generation.

Answer:

Introduction. In a basic encoder–decoder (sequence-to-sequence) model the encoder compresses the entire input sentence into **one fixed-length context vector**, from which the decoder generates the output. This **bottleneck** loses information and performance falls as sentences become longer. **Attention** (Bahdanau et al., 2014) removes it: instead of one vector, the decoder is given access to **all encoder hidden states** and learns, at every output step, **which input tokens to look at** – mimicking how a human translator glances back at the relevant source words.

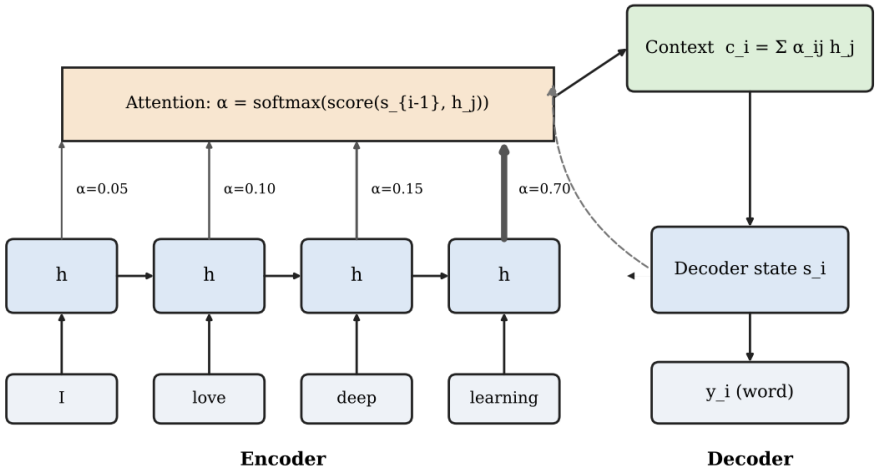


Fig. 14: Attention – the decoder computes weights α over encoder states and builds a context vector for each output word

Working – step by step (generating output word y_i)

- **1. Encode:** a (Bi)RNN/LSTM reads the input tokens and produces one hidden state $\mathbf{h_j}$ per token.
- **2. Score:** compare the previous decoder state s_{i-1} with every encoder state: $\mathbf{e_{ij} = a(s_{i-1}, h_j)}$. Additive form: $e_{ij} = v^T \tanh(W_s s_{i-1} + W_h h_j)$; multiplicative form: $e_{ij} = s_{i-1}{}^T h_j$.
- **3. Normalise (softmax):**

$$\alpha_{ij} = exp(e_{ij}) / \Sigma_{k=1..T} exp(e_{ik}),\ \alpha_{ij} \geq 0,\ \Sigma_j \alpha_{ij} = 1$$

- **4. Context vector:** $\mathbf{c_i = \Sigma_j \alpha_{ij} h_j}$ – a weighted sum, dominated by the most relevant tokens.
- **5. Decode:** $s_i = f(s_{i-1}, y_{i-1}, c_i)$ and $p(y_i) = \text{softmax}(W[s_i; c_i] + b)$. The context vector is **recomputed at every output step**, so the focus moves along the input.
- **6. Train:** the whole system, including the alignment function a(·), is trained end-to-end by back-propagation using the translation/generation loss – no explicit alignment labels are needed.

How the decoder "focuses"

In Fig. 14 the decoder is about to emit the word corresponding to "learning"; the learned weights are α = (0.05, 0.10, 0.15, **0.70**), so 70 % of c_i comes from $h_{learning}$ and the other words contribute little. For the next word the weights change, so the decoder attends to a different token. In translation this learns soft **word alignments** (including re-ordering between languages).

Types and benefits

Type	Idea
Additive (Bahdanau) / multiplicative (Luong)	Different score functions a(s, h)
Global vs local	Attend to all source positions vs a window around a predicted position
Self-attention (Transformer)	Every token attends to all tokens in the same sequence: Attention(Q,K,V) = $\text{softmax}(QK^T/\sqrt{d_k})V$ – removes recurrence, enables parallel training

- Solves the fixed-vector bottleneck → much better on **long sentences**; shorter gradient paths from output to every input token.
- **Interpretability** – attention weights can be visualised as an alignment matrix.

- Applications: machine translation, summarisation, question answering, image captioning, speech recognition.

MODULE 5 · Q.15

Summarize the significance of the Area Under the ROC Curve (AUC) in model evaluation.

Answer:

Introduction. For a binary classifier the output $\hat{y} \in [0, 1]$ is converted into a class by a **threshold** (normally 0.5). The ROC (Receiver Operating Characteristic) curve – originally developed in World War II to assess radar operators' judgement – plots the **True Positive Rate (TPR)** against the **False Positive Rate (FPR)** for **every possible threshold**; the **Area Under this Curve (ROC AUC)** summarises it as a single number.

$$TPR = TP / (TP + FN),\ FPR = FP / (FP + TN)$$

Calculation (textbook example – four predictions)

Actual y = (0, 1, 0, 1) and predicted $\hat{y}$ = (0.3, 0.5, 0.6, 0.9). Taking each $\hat{y}$ in turn as the threshold (a prediction above it is positive):

Threshold	TP	FP	TPR	FPR
≤ 0.3 (all positive)	2	2	1.0	1.0
0.3	2	1	1.0	0.5
0.5	1	1	0.5	0.5
0.6	1	0	0.5	0.0
0.9 (all negative)	0	0	0.0	0.0

Plotting (FPR, TPR) points and joining them gives the ROC curve in Fig. 15. Area = 0.5 × 0.5 + 0.5 × 1.0 = 0.25 + 0.50 = **0.75** (area under the curve by the trapezoid rule).

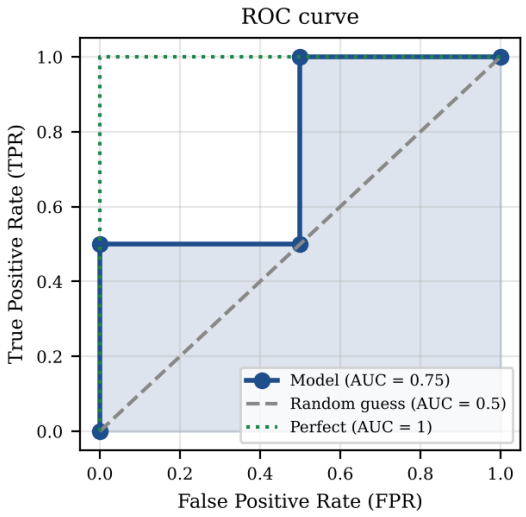


Fig. 15: ROC curve for the example (AUC = 0.75)

Interpretation

- **AUC = 1.0** – perfect classifier; **AUC = 0.5** – diagonal line, no better than flipping a coin; **AUC < 0.5** – worse than random (predictions inverted).
- Probabilistic meaning: AUC is the probability that the model gives a **randomly chosen positive example a higher score than a randomly chosen negative one**.
- In practice a perfect 1.0 is rarely attainable because every problem has an inherent ceiling of attainable AUC.

Significance in model evaluation

- **Blends two metrics** – TPR and FPR (i.e. both false negatives and false positives) – into one summary value.
- **Threshold-independent:** evaluates the classifier across the whole range $\hat{y}$ = 0 to 1, whereas accuracy judges only one threshold (e.g. $\hat{y}$ = 0.48 vs 0.51 are classed differently although the model's confidence barely differs).
- **Model comparison:** a single number to compare architectures (e.g. dense 92.9 %, convolutional 96.1 %, LSTM 92.8 % in the sentiment study); also guides choosing an operating threshold that trades TPR against FPR.

- **Less sensitive to class imbalance** than accuracy, because TPR and FPR are computed within each class.
- **Limitation:** for highly imbalanced data the precision–recall curve can be more informative; AUC does not tell which threshold to use or the cost of errors.

MODULE 5 · Q.16

Illustrate the process of generating word embeddings from a cleaned text corpus and the purpose of the embedding parameters.

Answer:

Introduction. A **word embedding** (word vector) represents each word as a dense, low-dimensional real vector so that words with similar meaning lie close together in the vector space. It is based on the idea *"you shall know a word by the company it keeps"*: the meaning of a word is represented by the words that tend to occur around it. **word2vec** (Mikolov et al., 2013) is an **unsupervised** technique – it needs no labels, so any corpus can be used.

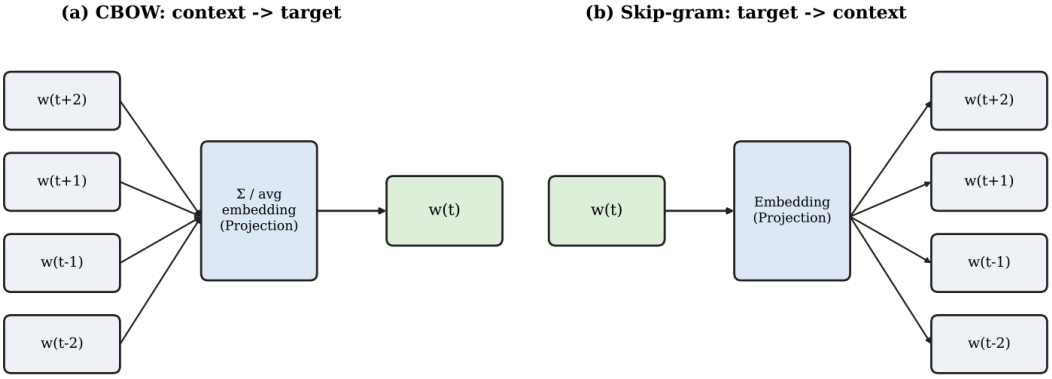


Fig. 16: word2vec architectures – CBOW predicts the target word from context; skip-gram predicts context words from the target

Process of generating embeddings

- **1. Pre-process the corpus** into *clean_sents* (tokenise, lowercase, remove stop words/punctuation, stem, bigrams).
- **2. Initialise** a vector for every vocabulary word at a **random** location in d-dimensional space (so each run gives different vectors, but similar relationships).
- **3. Slide a context window** over the corpus one word at a time. In the sentence *"you shall know a word by the company it keeps"* with target *word* and window 3, the 3 words on each side are context words.
- **4. Predict. Skip-gram (SG):** predict context words given the target. **CBOW:** predict the target from the average of all context words in the "bag" (order ignored; the window is "continuous").
- **5. Learn by SGD:** the prediction error is back-propagated and the locations of words are shifted a little, gradually improving the estimates; repeat over several iterations (epochs) of the corpus.
- **6. Output:** the trained embedding matrix – one vector per word (e.g. 64-D), with semantically similar words nearby.

```
from gensim.models.word2vec import Word2Vec
model = Word2Vec(sentences=clean_sents, size=64, sg=1, window=10,
                 iter=5, min_count=10, workers=4)
model.wv['dog']        # 64-dimensional vector for 'dog'
```

Purpose of the embedding parameters

Parameter	Purpose / guideline
sentences	Corpus as a list of lists (sentences → word tokens)
size	Number of dimensions of the vector space (hyper-parameter, vary by powers of 2). Doubling dimensions doubles downstream model cost; 64 was found better than 32, 128 gave no gain
sg	1 = skip-gram, 0 = CBOW (default). SG: better for small corpus, represents rare words well; CBOW: several times faster, slightly better for frequent words, good for very large corpus
window	Number of context words on each side: SG ≈ 10 (20 context words), CBOW ≈ 5
iter	Passes over the corpus (like epochs); default 5; more helps small corpora, too costly for huge ones
min_count	Minimum frequency for a word to be embedded (≈ 10); rare words have too few contexts to place reliably; larger value = smaller vocabulary
workers	Number of CPU cores used for parallel training

Alternatives

GloVe (count-based, global co-occurrence statistics, easily parallelised) and **fastText** (sub-word vectors; handles rare/out-of-vocabulary words) give comparable embeddings. For the Gutenberg example, 10,329 tokens occurring ≥ 10 times were embedded.

MODULE 5 · Q.17

Demonstrate the use of word embedding methods on a cleaned text corpus.

Answer:

Introduction. Once the corpus has been cleaned (*clean_sents*) and word2vec has been trained, the resulting word-vector space can be **inspected (intrinsic evaluation)** and **used as input to deep-learning models (extrinsic use)**. The demonstration below follows the Project Gutenberg example (64-dimensional vectors).

Step 1 – Train the model on the cleaned corpus

```
model = Word2Vec(sentences=clean_sents, size=64, sg=1, window=10,
                 iter=5, min_count=10, workers=4)
model.save('clean_gutenberg_model.w2v')
len(model.wv.vocab)        # 10,329 tokens occur at least 10 times
```

Step 2 – Look up a vector

`model.wv['dog']` returns the location of "dog" in 64-D space – a dense vector of 64 real numbers (a numeric representation usable by any neural network).

Step 3 – Similarity queries (cosine similarity)

```
model.wv.most_similar('father', topn=3)
# [('mother', 0.826), ('brother', 0.728), ('sister', 0.718)]
model.wv.doesnt_match('mother father sister brother dog'.split())   # 'dog'
model.wv.similarity('father', 'dog')                                # 0.44
```

Test word	Most similar word	Cosine similarity
father	mother	0.82
dog	puppy	0.78
eat	drink	0.83
day	morning	0.76
ma_am	madam	0.85

("ma_am" exists only because of the bigram step of pre-processing.)

Step 4 – Vector arithmetic (analogies)

```
model.wv.most_similar(positive=['father','woman'], negative=['man'])    # top: mother
model.wv.most_similar(positive=['husband','woman'], negative=['man'])  # top: wife
```

That is, $v_{father} - v_{man} + v_{woman} \approx v_{mother}$: directions in the space encode meaning (gender, tense), as in Fig. 17.

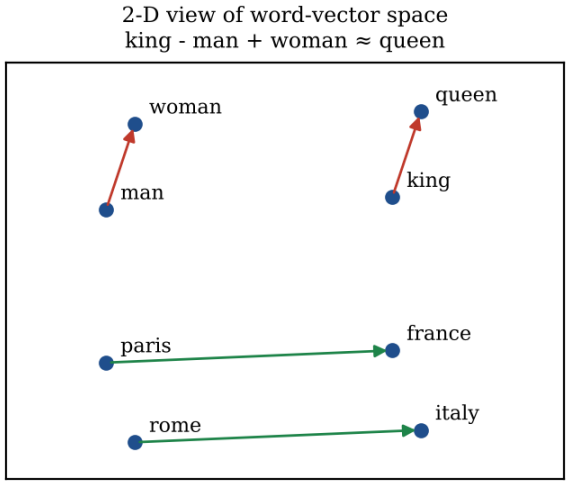


Fig. 17: 2-D view of word-vector space – parallel arrows show consistent relations (man→woman, king→queen)

Step 5 – Visualise using t-SNE

```
from sklearn.manifold import TSNE
tsne = TSNE(n_components=2, n_iter=1000)
X_2d = tsne.fit_transform(model.wv[model.wv.vocab])
coords_df = pd.DataFrame(X_2d, columns=['x','y']); coords_df['token'] = model.wv.vocab.keys()
```

The 64-D vectors are reduced to 2-D (t-SNE, van der Maaten & Hinton) and plotted (e.g. interactive bokeh scatter plot); related words (family terms, verbs, places) form clusters.

Step 6 – Use the embeddings in a model

- Feed word vectors (via an *Embedding* layer, initialised with these vectors or learned) into dense, CNN or LSTM layers for tasks such as sentiment classification, translation, search.
- Compared with one-hot vectors they are low-dimensional, capture similarity, and need less data for the downstream task.

MODULE 5 · Q.18

Apply sequential neural network architectures to develop solutions for sequence data and compare their strengths.

Answer:

Introduction. Sequence data – natural language, speech, sensor or financial time series – has order and variable length. The module's case study is a **binary sentiment classifier for IMDB movie reviews**: each review (word-index sequence) passes through an **Embedding layer** that converts words to vectors, then through a sequence-processing block, and finally a dense sigmoid output neuron.

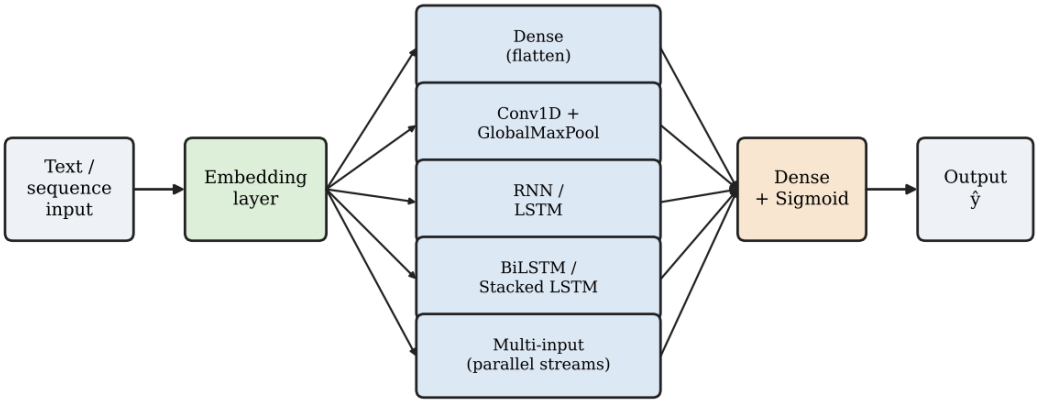


Fig. 18: One pipeline, interchangeable sequence-processing blocks

Architectures and how they are applied

- **Dense network (baseline):** Embedding → Flatten → Dense(ReLU) → Dense(sigmoid). Treats the review as a bag of positioned words; no notion of order.
- **Convolutional (Conv1D):** filters of length k (k-grams) slide over word vectors; global max-pooling keeps the strongest activation – learns word patterns that predict the sentiment, e.g. "not good".
- **Simple RNN:** $h_t = \tanh(Wx_t + Uh_{t-1})$; remembers order but suffers from vanishing gradients.
- **LSTM / GRU:** gated memory cells retain information over long spans.
- **Bidirectional LSTM:** reads the review forwards and backwards – context from both directions.
- **Stacked (deep) Bi-LSTM:** several recurrent layers (return_sequences=True) – more abstract temporal features.
- **Non-sequential (Keras functional API) – Multi-ConvNet:** parallel convolutional streams with different filter lengths (k = 2, 3, 4) whose outputs are concatenated before the dense layers; also Conv-LSTM hybrids.

```
model = Sequential()
model.add(Embedding(n_unique_words, n_dim, input_length=max_review_length))
model.add(Bidirectional(LSTM(n_lstm, dropout=0.2)))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
```

Comparison (validation ROC AUC from the textbook experiments, Table 11.6)

Model	ROC AUC (%)	Strength / remark
Dense	92.9	Simple, fast baseline; ignores word order
Convolutional	96.1	Fast to train; detects local word patterns; strong on this data
Simple RNN	84.9	Models order but forgets long context; weakest
LSTM	92.8	Long-term memory via gates; slower (sequential)
Bi-LSTM	93.5	Past + future context
Stacked Bi-LSTM	94.9	Deeper features; best among recurrent models
GRU	93.0	Fewer parameters than LSTM, similar accuracy
Conv-LSTM	94.5	Local patterns from CNN + sequence memory from LSTM
Multi-ConvNet	**96.2**	Parallel multi-width filters; best model (val. accuracy 89.4 %)

Discussion

- Convolutional and Multi-ConvNet models won here because the sentiment is carried by short phrases and the data set is modest; the authors suggest Bi-LSTMs might win on a much larger data set.

- Choose **RNN-family** models when long-range order matters (translation, speech); **CNN** for speed and local patterns; **parallel/hybrid** architectures to combine strengths.
- Use dropout, early stopping (best epoch had the lowest validation loss), and ROC AUC for evaluation.